



Lab 5

x86-64 Assembly: Stack Overflow

Buffer Slide

Discussing doubts/solutions attempts for last week's inlab / labexam / outlab

Outlabs 3 & 4 are out

All previous labexam solutions are uploaded on the repo



Plan for this lab

- How to handle **decimal** numbers and **Floating point** operations
- Breaking code into **functions** and understanding the **stack**
- **Syscalls** : Use some predefined functions which can be used to call the OS



XMM Registers

These are special 128 bit registers used for floating point arithmetic (wired up to the parts of the cpu which have capabilities of performing floating point arth)

- **xmm0-xmm15** are the 16 xmm registers present in 64 bit systems. (In 32 bit systems only 8 xmm registers are present)
- **SSE Instruction** are special instructions that are present in the ISA to perform operations with these registers.
- **So how do we use them ?**

```
add_cplx:
```

```
    movsd xmm0, [rdi+8]
    movsd xmm1, [rdi+16]
    addsd xmm0, [rsi+8]
    addsd xmm1, [rdi+16]
    movsd [rdx+8], xmm0
    movsd [rdx+16], xmm1
    ret
```

Basic xmm/SSE instructions

movsd xmm, xmm/*mem	Can move data to / from memory location or other xmm
movsd xmm/*mem, xmm	
addsd xmm, xmm/*mem	add / subtract, in place
subsd xmm, xmm/*mem	
mulsd xmm, xmm/*mem	multiply / divide in place
divsd xmm, xmm/*mem	
movq xmm, reg/*mem	Can move data to / from general purpose register
movq reg/*mem, xmm	



We are only using **64 bits** out of the **128 bits** width in xmm registers

It's inlab (1) time



Best Functions Forever <3

- **call <label>**: Pushes the address of the **next** instruction (also known as the return address) and jumps to the label.
- **ret**: Pops the return address from top of the stack and jumps back to respective address

```
main:
    call my_func
    ; execution resumes here

my_func:
    ; do work
    ret
```

Careful what you wish for

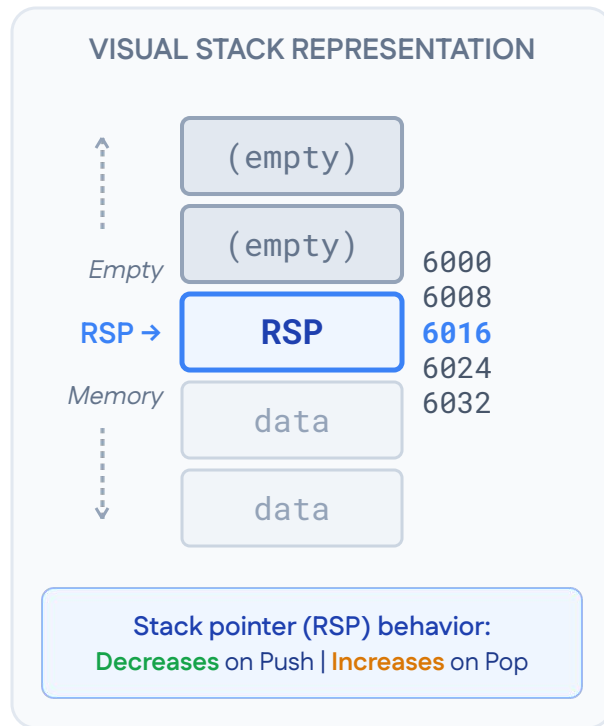
- What if we push and pop something else on the stack before doing ret?
- ret will read **something else** and it will not return :(



What is a stack?

- Recall the function stack mumbo jumbo in c++
- We push things on the stack before a function call
And pop them afterwards
- Stack grows downwards! (Higher -> lower memory addresses)
- Push operation: eg: `push rax`
- Equivalent assembly:
`sub rsp, 8 ; say initially rsp=6016`
`mov [rsp], rax`
- Pop operation: eg `pop rax`
- What should be equivalent assembly?

```
mov rax, [rsp]
add rsp, 8
```



Caller Callee Conventions

- **Caller saved registers:**

- **Function arguments:** rdi, rsi, rdx, rcx, r8, r9
- **Return value:** rax
- **Scratch:** r10, r11
- **All xmm registers:** xmm0-xmm15

- **Callee saved registers:**

- **General Purpose Registers:** rbx, rsp, rbp, r12, r13, r14, r15



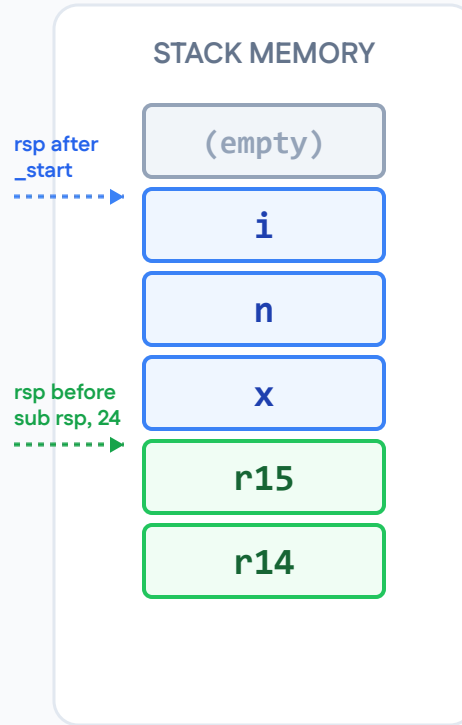
Example: Function

```
_start:
    push rbp
    push rsp
    push rbx
    push r12
    push r13
    push r14
    push r15

    sub rsp, 24

    mov rax, 0
    mov [rsp + 16], rax ; x = 0
    mov rax, 3
    mov [rsp + 8], rax ; n = 3

    mov rax, 0
    mov [rsp], rax ; i = 0
```



```
main.c

// Equivalent in C

long long x = 0;
long long n = 0;
long long i = 0;
```

Alignment is key

```
void example() {  
    char a = 'A'; // 1 byte (offset -1)  
                // 3 bytes of PADDING added here  
    int b = 42;   // 4 bytes (offset -8, must be 4-byte aligned)  
    char c = 'C'; // 1 byte (offset -9)  
                // 7 bytes of PADDING added here  
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)  
}
```

- Recall memory alignment of data types from the theory course
- TLDR: **sizeof(T) % alignof(T) == 0**
- This is a standard practice (called SysV ABI) that C/C++ compilers implement for us
- In x86 we will have to manually insert “padding” to achieve this
- **Padding** – leaving some gaps in the memory address space

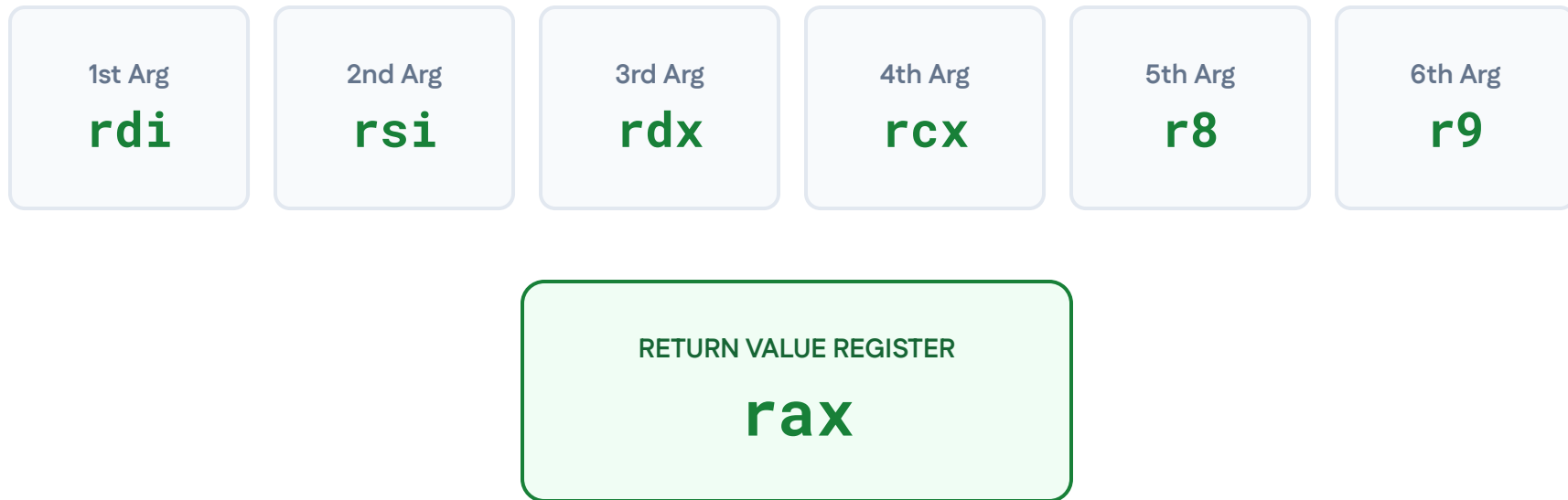
Alignment is key

```
void example() {  
    char a = 'A'; // 1 byte (offset -1)  
    // 3 bytes of PADDING added here  
    int b = 42; // 4 bytes (offset -8, must be 4-byte aligned)  
    char c = 'C'; // 1 byte (offset -9)  
    // 7 bytes of PADDING added here  
    double d = 3.14; // 8 bytes (offset -24, must be 8-byte aligned)  
}
```

- For eg: for `int b` -> starting address should be $\equiv 0 \pmod{4}$
- We insert padding suitable for this
- Remember xmm? Address should be 8 byte aligned while using `movsd` and `movaps/movapd` etc. Note: `movups` allows unaligned!
- When you call a function make sure its **16 byte aligned (at `call` boundary)**
- Especially when you use C functions like `printf` etc.. Why?

General functions and their convention

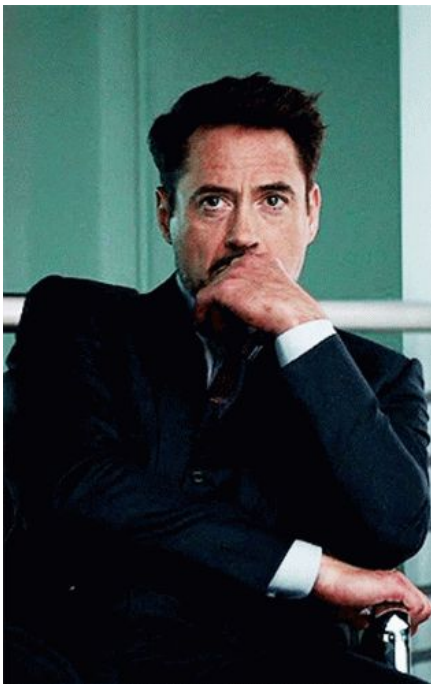
Register Arguments (Passed in Order)



Random questions

Common Doubts

- Why follow caller-callee convention?
- Why push/pop registers before & after every call?
- Why follow argument and return value conventions at all?



The Answer

MODULARITY

- Allows seamless integration with **C function libraries**.
- Enables using functions written by other people.
- Required to perform standard **system calls**.
- Mutual convention ensures code interoperability.

How do we use **C functions** in assembly?? (CHECK OUT OUTLAB)

Syscall me on my cell phone

What is a Syscall?

Requesting a privileged action from the OS (like writing to stdout).

The Mechanism

1. Load the syscall ID into **rax** (e.g., 1 for `sys_write`).
2. Load arguments using standard ABI rules (**rdi**, **rsi**, **rdx**).
3. Substitute **r10** in place of **rcx** if used
4. Execute the **syscall** instruction.

Position · Register · Description (sys_write Example)

Syscall Number **rax** (1 for **sys_write**)

1st Argument **rdi** (File Descriptor (1 for **stdout**) for write)

2nd Argument **rsi** (Pointer to buffer string (**msg**) for write)

3rd Argument **rdx** (Length of buffer (**msglen**) for write)

**Note: Register arguments (rdi, rsi, rdx, etc.) hold entirely different meanings depending on the specific syscall invoked.*

It's inlab (2 & 3) time

